

Software Evolution Metrics for the Detection of Trojan Code in **npm** Packages

Giacomo Benedetti¹, Luca Caviglione¹, Giovanni Lagorio², and Marco Zoratti^{1,2}

¹ Institute of Applied Mathematics and Information Technologies, Genova, Italy

`name.surname@ge.imati.cnr.it`

² University of Genova, Genova, Italy

`name.surname@unige.it`

Abstract

The increasing sophistication of attacks against the software supply chain imposes the need for lightweight and effective countermeasures. The recent threat RATatouille targeting the **rand-user-agent** package of the **npm** ecosystem has demonstrated that attackers can stealthily embed trojans directly into source code by exploiting “visual deceiving” techniques. Therefore, this paper focuses on the detection of trojan injection mechanisms leveraging bursts of blank spaces and homoglyphs to evade manual inspection and traditional countermeasures. To perform tests, we compiled a dataset consisting of the 2,800 most popular **npm** packages and we considered seven classic metrics used to describe the evolution of the code as well as three ad-hoc indicators. Our results showcase that generic metrics are inadequate for fast-evolving ecosystems, whereas ad-hoc indicators can help to spot the presence of trojans cloaked within the source code.

1 Introduction

Mitigating the impact of malicious software has been a prime security concern for more than two decades. During the years, researchers and industrial experts proposed many countermeasures against malware, such as detectors based on signatures, static analysis tools, reverse engineering frameworks, and sandboxing environments [2, 9]. The recent surge of AI ignited the proliferation of advanced approaches, e.g., graph neural networks for classifying malicious software [5, 13], and LLMs for anticipating offensive trends [10]. However, better countermeasures triggered an “arms race” leading to a composite panorama of defensive tactics and practices [6, 18].

To evade detection, attackers regularly use advanced techniques ranging from information hiding to obfuscation [17]. Several elusive mechanisms leverage a multi-stage loading blueprint, where a monolithic malware is decoupled into different components, e.g., dropper and payload. Due to the growing diffusion of third-party packages and the ubiquitous adoption of open-source software, a major offensive template aims at injecting malicious contents in popular repositories [12]. Many recent threat campaigns targeted software packages for which the attacker already obtained the access, e.g., via social engineering or fake identities [16]. The threat actor could also create a “rogue” dependency for importing the malicious payload when merged against the target project. Despite the used template, the aim of the attacker is to avoid the detection by leveraging the burden of source code, commits, and turnovers of maintainers, which are often difficult to assess and model even with the aid of AI [8].

The injection of trojan code in a package requires deceiving the developer to hinder the identification of the unwanted payload. In general, attackers take advantage of two major techniques. The first exploits *blank spaces* for shifting the malicious payload outside the visible area of the development environment, e.g., the editor or the diff console. The second exploits *homoglyphs* preventing the developer to recognize additional functions or variables due their

visual similarity with legitimate ones. A major example is the RATatouille malware that has been injected in the `rand-user-agent` package version 1.0.100 hosted by the `npm` ecosystem. Specifically, the attacker appended malicious instructions to a benign line of code interleaved with a trail of blank spaces [1]. Both blank spaces and homoglyphs are effective for evading manual code analysis practices. The “visual” ability of the human is highly impaired since many editors (e.g., the code renderer of the `npm` web interface) do not alert the developer nor automatically wrap lines. Mitigating this new-wave of *visual deceiving* attacks is then of prime importance, especially to prevent the need of deploying more complex approaches, such as dynamic analysis. Unfortunately, spotting subtle variations of the code could be a time- and resource-expensive process, especially for large open-source projects. A promising defensive approach entails *code evolution*, which aims at modeling the evolution of a package to identify alterations. For instance, [24] showcased a prime attempt of analyzing GitHub repositories to detect injections into the distributed software. However, the comparison is limited to files within the registry, thus may not be effective to spot additional statements added directly to the sources.

Therefore, this paper provides a first analysis on how metrics related to the software development life-cycle can enable the identification of trojan code injected via blank spaces and homoglyphs. To pursue this goal, we shortlisted seven metrics taken from the literature dealing with code analysis (e.g., number of lines of code) and designed three metrics for our threat model (e.g., blank space ratio). To assess the performance of our approach, we conducted tests on 2,800 most popular `npm` packages. We selected `npm` since is one of the most used and composite ecosystem at the basis of modern software supply chains [26]. Our results support the intuition that ad-hoc metrics are required to detect the injection of trojans in source code, whereas more general indicators are impaired by the fast development pace characterizing modern ecosystems.

Summing up, the contributions of this work are: *i*) a dataset containing evolution metrics for `npm` packages, and *ii*) a quantitative evaluation on whether they can identify blank spaces and homoglyphs attacks.

The rest of the paper is structured as follows. Section 2 provides background information and the motivations, while Section 3 reviews the literature. Section 4 introduces the threat model, Section 5 outlines the investigation methodology, and Section 6 discusses a brief characterization of `npm` packages. Section 7 showcases numerical results and Section 8 concludes the paper.

2 Background and Motivation

This section introduces basic concepts and motivates the need of new detection approaches. Specifically, Section 2.1 provides an overview on how open-source packages can be analyzed to identify malicious activities. Section 2.2 briefly outlines main malware detection techniques and why new metrics are needed to face trojan injections.

2.1 The Open-Source Development Process

Modern software supply chains heavily rely on open-source ecosystems, where packages evolve through a continuous stream of releases, contributions, and automated updates [21]. In ecosystems such as `npm`, the development workflow is typically mediated by version control services (e.g., GitHub) and driven by a composite set of actors, including maintainers, external contributors, and bots handling dependency updates or maintenance tasks. The evolution of a package is then captured by the *commit history*, which could be composed of more than thousands of

entries for the most popular projects [26]. Despite differences in contributor roles and commit styles, software projects tend to preserve structural regularities that emerge from coding conventions and languages. Examples include limiting the maximum line length for readability, consistent formatting practices, and the stable use of a specific text encoding across the code-base [20]. The major behaviors that emerge across the commit history form an *evolutionary signature* for the project.

The ability of computing a baseline behavior becomes particularly relevant in the context of threat detection since it allows to recognize subtle anomalies that may surface during the development process. This is especially useful to identify attackers seeking to compromise repositories by “blending” their malicious modifications within legitimate software [16]. For instance, the introduction of a line hiding a payload after an excessive sequence of blank spaces, or the appearance of identifiers encoded with characters outside the usual encoding range, could be captured by analyzing deviations from the “historical” behavior of the project. Hence, software evolution metrics can serve as lightweight indicators to spot out-of-pattern modifications without requiring a deep code analysis. This is critical for ecosystems like `npm`, where the number and frequency of releases make manual inspection impractical and where the development pace favors attackers exploiting visual ambiguity or formatting-based evasion techniques [25].

2.2 Limitation in Malware Detection Strategies

Despite being usually very effective, classic detection methods tend to struggle when adversaries compromise the source code itself. This is why, an increasing number of threat actors targets the software supply chain and injects trojans directly into upstream repositories [24]. When the malicious payload becomes part of the official release, traditional security mechanisms may overlook the hazard, as the artifact originates from a legitimate maintainer.

Software supply chain threats like trojan source attacks exacerbate this problem by exploiting discrepancies between what humans see and what compilers interpret [3, 4]. Techniques such as the insertion of long sequences of blank spaces or the use of homoglyphs allow the concealment of instructions/compounds or introduce deceptive function names that visually resemble legitimate ones. Static analysis tools often normalize blank spaces or character encodings, causing to discard the features that can reveal an additional payload [3]. Dynamic analysis could be insufficient since malicious payloads may remain dormant, require specific activation conditions, or serve as exfiltration helpers embedded in apparently harmless functions. Moreover, the volume and churn of modern ecosystems make dynamic inspection resource-intensive and impractical at scale [14]. The analysis of code evolution then offers a strategy that can complement classic static/dynamic malware detection techniques. Rather than understanding the semantics of the injected malicious logic, the idea is to focus on measurable deviations from the historical development patterns of the project.

3 Related Work

The security of package registries, such as `npm` and `PyPI`, has been extensively investigated in recent years due to the surge in supply chain attacks. The fragility of the `npm` ecosystem has been already highlighted in [27] by showing how individual maintainer accounts act as single points of failure for thousands of dependent packages. A curated dataset of malicious open-source `Node.js` packages, which have been used in real-world attacks, has been provided in [19]. More recently, this view has been expanded by proposing a general taxonomy independent of a specific programming language or ecosystem [16]. While these works focus on identifying

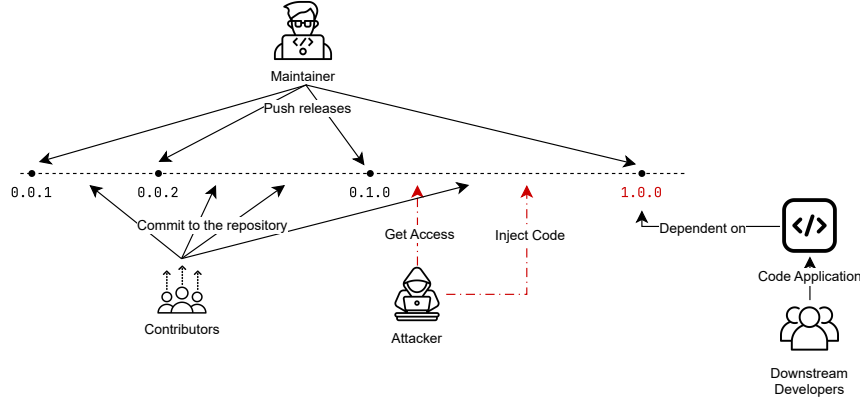


Figure 1: Threat model considered in this work.

malicious packages via metadata (e.g., author reputation and package naming patterns) or dependency graphs, our work focuses on detecting the malicious payload itself embedded within the source code during the maintenance life-cycle.

As regards “visual deceiving” vulnerabilities like *trojan source*, they have been formally introduced in [3]. In essence, the authors demonstrated how Unicode control characters can be exploited to reorder tokens in source code to deceive human reviewers, while remaining valid for compilers. This class of attacks exploits the “visual gap” in development tools: while IDEs and diff viewers render code for readability, compilers process the underlying raw byte stream. Although compilers can give warnings when they encounter these kind of patterns, attackers can circumvent these countermeasures by using homoglyphs or, as highlighted in the recent RATatouille campaign, by exploiting widely accepted formatting characters, such as blank spaces, to push malicious logic out of the visible viewport [1]. Unlike general obfuscation for thwarting reverse engineering [17], visual spoofing attacks are designed to survive human-driven code review processes.

Detecting malware through software metrics is a well-established practice. However, within the context of high-level interpreted languages like JavaScript, these metrics can be ineffective due to *minification* and legitimate code density. Previous work proposed the use of repository history to identify anomalous commits [24], a concept that aligns with our methodology. Nevertheless, the approach primarily targets file additions or metadata changes, while we address suspicious source code changes. However, a relevant corpus of research on security of open-source ecosystems focuses on package naming attacks, which are exploited via two main prevalent techniques. The first is *typosquatting* and relies on users making typographical errors when issuing installation commands (e.g., omitting a hyphen or swapping two characters). This causes to download malicious packages instead of the intended ones, and allows the attackers to execute code on the machine of the victim. Typosquatting may leverage a wide-range of cases, for instance the malicious code could be cloaked in test cases or in pre-/post-install scripts [11, 19]. The second technique is *combosquatting* and creates plausible-sounding package names by combining a familiar target name with legitimate keywords (e.g., **analytics**, **proxy**, or **helper**) to deceive developers into trusting a rogue dependency [15].

Attacks based on “squatting” target the distribution phase by mimicking package identifiers, while our work addresses threats injected directly into source code artifacts. Thus, our goal is to introduce lightweight evolution metrics, specifically focusing on blank space ratios and

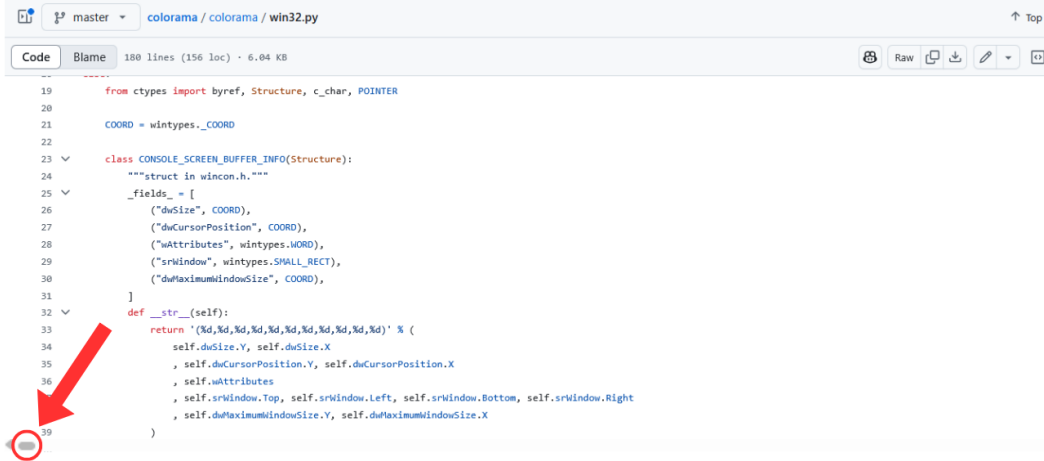


Figure 2: Example of blank spaces used to hide a malicious payload outside the editor frame.

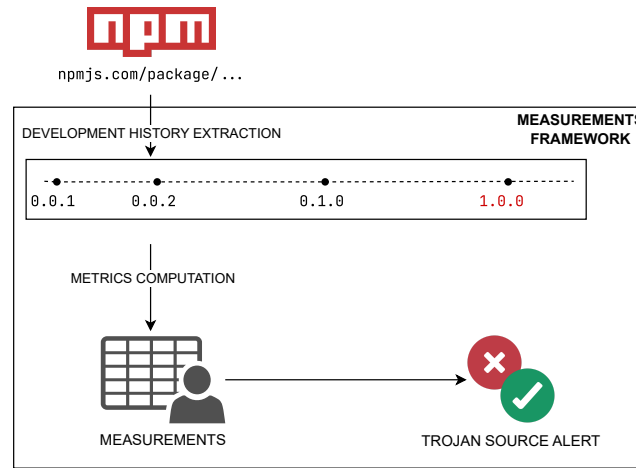
encoding variations.

4 Threat Model

The *trojan source* umbrella refers to a hiding method that injects characters into the source code to force text editors to display a content that differs from what compilers and interpreters process [3, 4]. Figure 1 showcases the threat model considered in this work. In more detail, the attacker hijacks a code repository and injects malicious code at a specific point in time, resulting in the malicious payload being included in the release made by the project maintainer. Since the attacker applies a minimal modification using a trojan source technique, the injected code is hardly detectable.

In this work, we consider an attacker injecting code via two major templates, i.e., blank spaces and homoglyphs. The first class consists of an attacker capable of appending a one-line malicious payload after a burst of blank space characters. The assumption here is that the developer uses an editor without any active line/text wrap functionalities, i.e., it does not automatically apply carriage returns to fit the text in the editor window. Figure 2 showcases how the online editor of GitHub can be exploited to decoy the user. As shown, the attack only alters the size of the horizontal bar for managing the area rendering the code. If this detail is missed, the developer will fail to spot that a payload has been cloaked outside the visible area. The second class allows the attacker to deceive a developer to use a malicious function via a homoglyph. Figure 3 showcases an example where a package has been hijacked. The code has been altered to invoke a “bogus” version of the `read_csv` function instead of the one implemented in the `pandas` package. This has been made possible since the Unicode characters U+0065 and U+0435 are rendered in a similar manner, i.e., the ASCII and cyrillic ‘e’ are visually equivalent. To not disclose the attack, the injected `read_csv` mimics all the functionalities of the original code. Yet, it also contains some commands added by the attacker, which have been highlighted with a red box in the example of Figure 3. Downstream developers could then be misled into using the homoglyphed `read_csv` function via poisoned code-completion features or fake examples that can be copied/pasted into the editor.

We point out that, both the blank space and the homoglyphs techniques are effective only

Figure 3: Example of a homoglyph used to invoke an alternative version of `read_csv()`.Figure 4: Methodology used for collecting and analyzing data from the `npm` registry.

when a shallow analysis of the code is applied. In fact, more deep techniques (e.g., dynamic analysis) may easily reveal their usage. However, the complexity of the supply chain that makes packages available to developers hides the distribution of potentially infected artifacts.

5 Investigation Methodology

This section discusses our approach for evaluating the evolution of the `npm` ecosystem. In more detail, Section 5.1 introduces the framework used for collecting the needed information, while Section 5.2 describes the metrics for modeling the evolution of `npm` packages. Lastly, Section 5.3 describes how malicious samples have been prepared.

5.1 Measurement Framework

To collect information of real `npm` packages and to evaluate the feasibility of detecting trojan code injections via metrics describing the code evolution, we implemented a set of command

Table 1: Metrics used to spot trojan code (**bold**: metrics effective for our threat model).

Metric name	Brief description
Maximum line length	Longest sequence of characters without a line break in a file.
Blank space ratio	Fraction of blank space characters over total characters.
Number of encodings	Count of characters with a different encoding observed across files.
Commit frequency	Intervals when commits happen from contributors in the projects.
Commit time	Date and time of the commit.
Cyclomatic complexity	Complexity of the code according to the cyclomatic algorithm.
Line of code added per release	Lines of code added in a release.
Code size in bytes	Size of the project in bytes.
Number of changed files	Difference between the total number of files in consecutive releases.
Shannon entropy	Amount of information contained in a file.

line tools written in Python¹. Figure 4 showcases the overall framework, which relies on three stages: extraction of evolutionary data, computation of the selected metrics, and assessment of anomalies arising across the life-cycle of a package.

As a first step, our measurement framework interacts with the `npm` registry to reconstruct the development history for a given package. To this aim, a suitable function retrieves all tagged releases: by relying upon *tags* rather than individual commits, our approach reduces the burden caused by temporary or experimental changes. The collected information for a package contributes to a clearer view of the evolution patterns relevant to downstream users. As a second step, the source files composing a single release are “sanitized”. Our tool removes minified, auto-generated, or test artifacts that would skew the measurements. The remaining source files are then parsed for computing the various evolution metrics. Obtained data captures how the package evolves over time and allows to establish a baseline describing its expected behavior. Our framework can iterate through a list of packages for computing a `csv` file with measurements and indications on potential trojanized code. However, performing automatic detection is outside the scope of this paper and is part of our ongoing research.

5.2 Selection and Design of Metrics

The selection and design of suitable indicators for assessing the evolution of `npm` packages are driven by two objectives. Metrics should be general enough to effectively capture the evolution of the code base but they should also guarantee a sufficient sensitivity against the (subtle) alterations characterizing the use of blank spaces or homoglyphs. To this aim, we shortlisted a set of ten indicators, summarized in Table 1.

Specifically, we considered the *commit frequency* to identify potential discrepancies in the habits of developers that may suggest a malicious behavior. The *commit time* may indicate one or more commits at an unusual time, e.g., an attacker residing in a different timezone. The *cyclomatic complexity* can capture code with an alternative form compared to regular contributions, such as a malicious payload implementing advanced offensive techniques. The *lines of code* added across the history of the package can be suitable to identify the presence of blank lines to push contents in a portion of the file “difficult” to inspect. The *code size* in bytes can determine whether a malicious payload is present, whereas the *number of changed*

¹To reproduce results presented in the paper and to conduct further research, the repository containing code, scripts, and the complete dataset is publicly available online at: <https://github.com/marchacio/ITASEC26-source-attack-metrics>


```

for (const S of R) {
  if (t[0].stopKey) {
    r.emit(
      'response',
      'HTTP upload stopped: ' +
      P +
      ' files succeeded, ' +
      Q +
      ' files failed\n',
      0
    )
  }
}

```

Figure 5: Part of the RATatouille trojan, deobfuscated and re-formatted for the sake of clarity.

files may indicate the introduction of ad-hoc files containing a trojan, configuration data, or supplemental attack stages. Lastly, the *Shannon entropy* can provide useful information to reveal the presence of obfuscated code, e.g., trojanized statements or compounds.

To complete these general indicators, we also introduce three metrics tailored for our threat model. The first is the *maximum line length*, i.e., the longest sequence of characters without a newline within any file of a release. This metric is effective for detecting malicious payloads appended after long bursts of characters, as commonly used in visual-deceiving attacks. The second is the *blank space ratio*, defined as the fraction of blank space characters over the total number of non-blank characters. Large and unexpected increases suggest spacing-based hiding strategies. The third is the *number of encodings* used within a file, which counts the occurrences of “uncommon” characters that may comprehend homoglyphs.

5.3 Payload Injection

To assess the metrics presented in Table 1 under fair conditions, we injected a real trojan in 100 randomly-chosen packages. In more detail, we modified the last tagged release of a package with the RATatouille malware, i.e., a one-line trojan designed to initiate a command-and-control channel on the host of the victim. Figure 5 reports a portion of the RATatouille payload, which has been deobfuscated and properly formatted for the sake of readability. As shown, the malware iterates over a list of files *R* defined by the attacker. Each item in the list is then handled via the `emit()` function triggering its upload to the infected machine. We selected such a class of trojans since they are characterized by a minimal footprint and allow us to model the operational simplicity of real software supply chain attacks, where the typical goal is to inject only the code necessary for persistence or exfiltration [22].

To inject the payload, we implemented the two variants of the threat model discussed in Section 4. For the case of blank space, the remote trojan has been added by appending its code after a sequence of 800 blank spaces. For the case of homoglyphs, we injected the trojan by modifying four characters in the respective homoglyphs, i.e., four Cyrillic ‘a’ (U+0430) instead of the Latin version (U+0061). For both cases, the injection of the payload has been done by randomly selecting both the line at which “insert” the payload and the targeted `.js` source file. The repository for the paper contains both scripts and payloads to create `npm` packages “poisoned” by the malicious payloads.

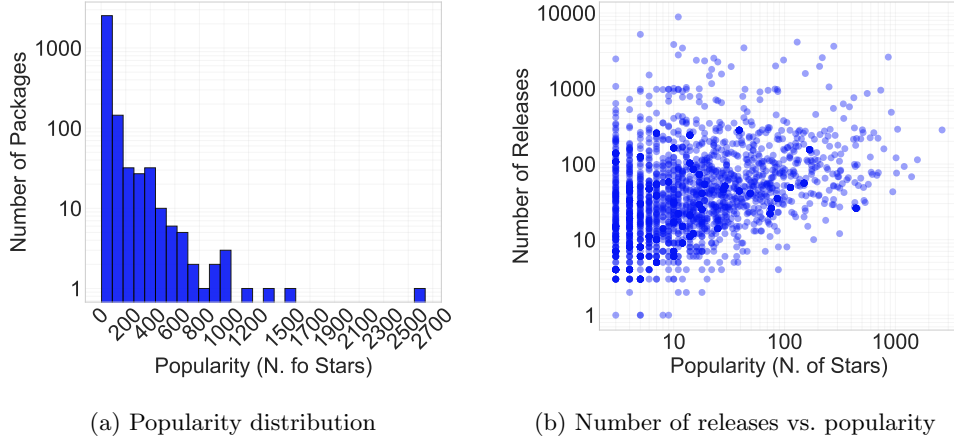


Figure 6: Popularity metrics of the considered 2,800-package dataset.

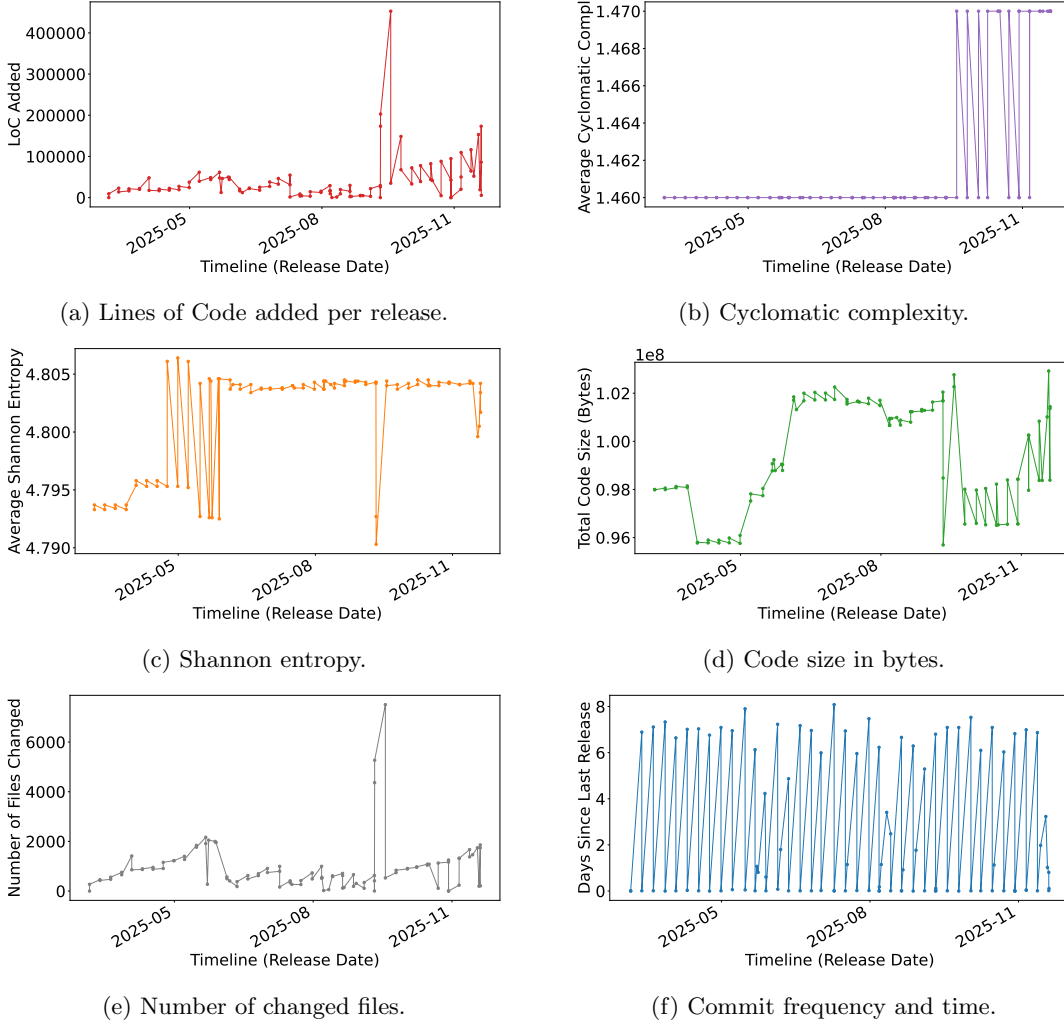
Table 2: High-level view of the dataset composed of 2,800 `npm` packages.

Versions per Package (Release)		Files per Release	
Pkg. with 1 version	20	Rel. with 1 file	44,276
Pkg. with < 10 versions	528	Rel. with < 20 files	275,727
Pkg. with > 50 versions	933	Rel. with > 20 files	89,801
Mean	145.45	Mean	171.44
Median	31.00	Median	4.00
Max	10,161	Max	44,709

6 Data Collection and Preliminary Analysis

To have a suitable dataset to evaluate both the feasibility and the performance of our approach, we collected the 2,800 topmost ranked `npm` packages from the `npm-rank` list [23]. Apart being representative of realistic open-source software used in a huge variety of settings, focusing on popular packages prevents to avoid data poisoning. In fact, many packages available in the `npm` registry are test or unfinished projects that do not represent actual packages. Resorting to 2,800 packages also allows for a suitable trade-off between the relevance of the result and the execution time. Specifically, our tests have been conducted on an Ubuntu 20.04 machine equipped with an Intel i9-9900KF CPU @3.60 GHz and 32 GB of RAM. On average, the complete analysis of a package release required 18 seconds.

Table 2 provides a high-level view of the computed dataset. As shown, the number of versions per package is generally high. However, the median is 31 versions, which hints at a dataset heavily influenced by a few packages with many versions. There is also a significant number of packages with more than 50 versions. This supports our intuition of trying to spot the presence of attacks via “evolutionary” information, i.e., the larger amount of evolutionary information can be derived, the better. The collected 2,800 packages also vary in the number of files: many packages have a small number of items, with a mean of 171.44 and a median of 4. Many releases have fewer than 20 files, hence our approach is manageable in terms of resources and execution time. Since the popularity plays a major role in the attractiveness of a package from the perspective of the attacker, Figure 6 showcases core popularity metrics computed over our dataset. In more detail, Figure 6a depicts the distribution of popularity as computed by the

Figure 7: Example of general metrics behavior on the last 100 versions of the **Angular** package.

official **npm** registry. As shown, the maximum number of stars is equal to 2,638 with a mean of 15.76. According to [27], the popularity of packages in our sample follows the same trend as the overall **npm** ecosystem, making our subset representative of mainstream development practices. Besides, Figure 6b shows that the number of releases for highly-popular packages (i.e., more than 100 stars) is clustered largely above 10 releases, indicating that our dataset is particularly suitable for being investigated through evolution metrics.

Before conducting large-scale tests, we performed a preliminary analysis to identify the most discriminative indicators for revealing the presence of trojans. As an example, Figure 7 depicts the results for the seven general metrics reported in Table 1 when computed on the last 100 releases of the **Angular** package also by considering a version containing the trojan. As shown, indicators such as the commit frequency, the cyclomatic complexity, or the code size turned out to be too sensitive to benign factors such as refactoring, dependency changes, or verbosity differences. For instance, version 19.2.5 of **Angular** reduced the dependence against

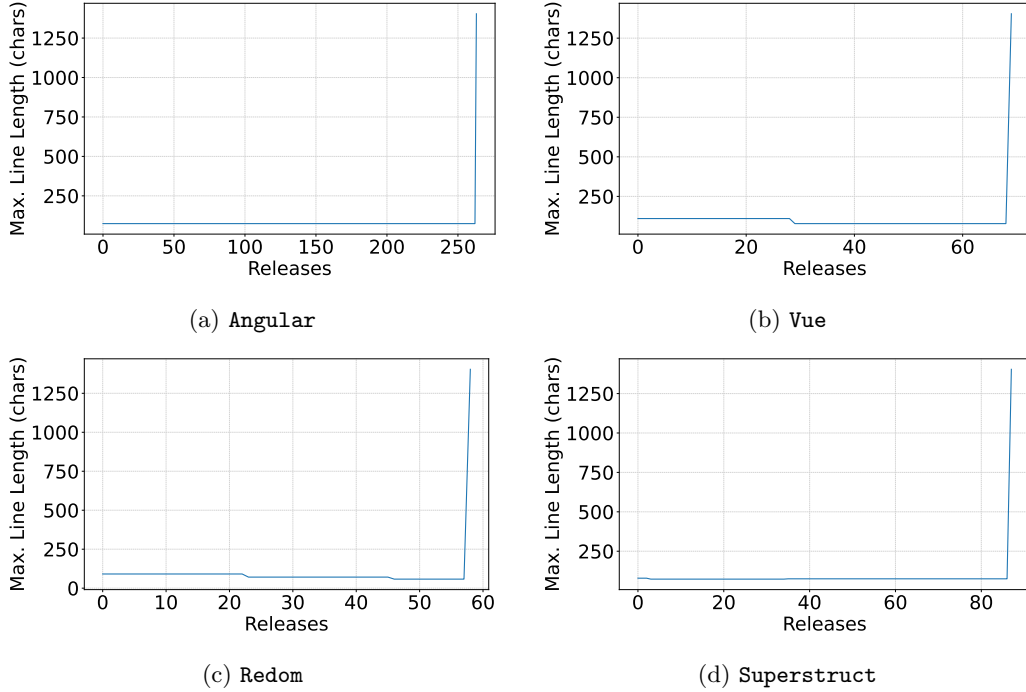


Figure 8: Maximum line length metric computed on four packages from our dataset.

NgModules functionalities, thus leading to a considerable fluctuation of the lines of code indicator. Alterations in indicators were then properly due to the refactoring of many functionalities of the **Angular** framework. In other words, general metrics do not provide a solid baseline for the identification of attacks carried out through the injection of blank spaces and homoglyphs. Therefore, the metrics designed for our threat model and reported in bold in Table 1 are expected to exhibit a stronger stability across the “normal” development process.

7 Results

This section presents the results obtained by considering 2,900 samples, i.e., the collected 2,800 **npm** packages and 100 injected packages. For the sake of clarity, we only report results for the blank space ratio, maximum line length, and number of encodings. The online dataset contains **.csv** files for each package-metric pair having the same layout. Specifically, rows correspond to the content of a **npm** package and columns correspond to versions. When an asset is not present in a specific version of the package, the “cell” is empty. In the following, Section 7.1 reports the results for blank spaces, while Section 7.2 for homoglyphs.

7.1 Blank Spaces

When computed against our dataset, the blank space ratio and the maximum line length exhibit a stable behavior. This could be ascribed to the coding convention of not exceeding 80 characters per line when editing textual contents [20]. The metrics should be then considered particularly informative for capturing anomalies associated with the injection of blank spaces.

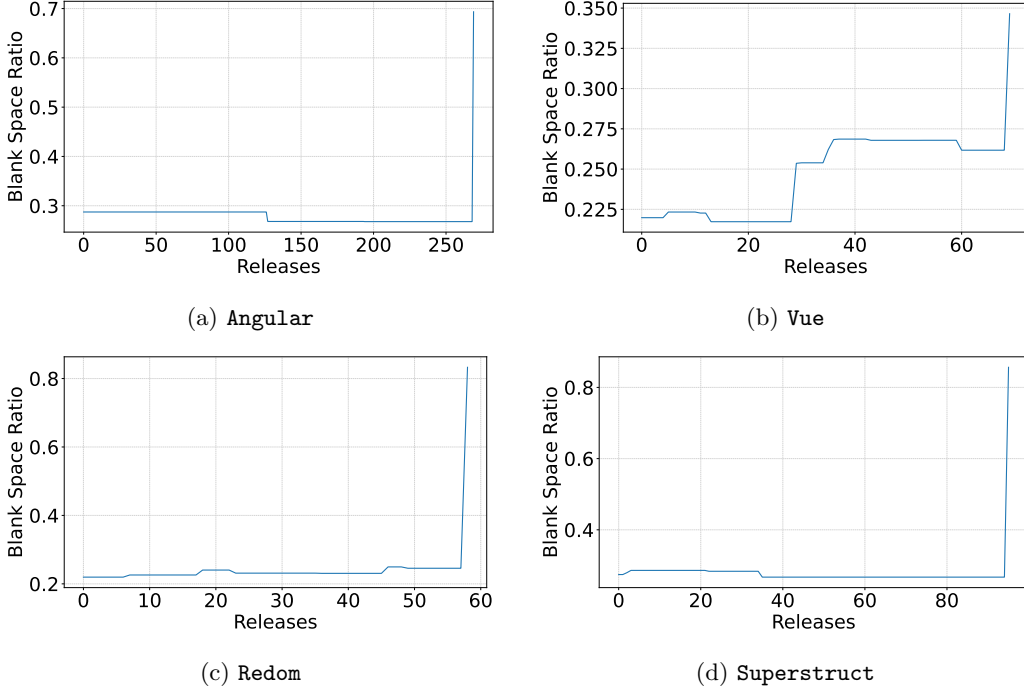


Figure 9: Blank space ratio metric computed on four packages from our dataset.

To make some examples, Figure 8 portrays the behavior of the maximum line length for four representative packages in our dataset that have been targeted by the trojan. As shown, the metric exhibits similar trends regardless of the development process or the package type. For instance, **Angular** with more than 250 releases and **Redom** with less than 60 releases are both consistently under 100 characters per line, as reported in Figure 8a and Figure 8c, respectively. As a result, the presence of a release poisoned with a trojan will make the metric to vastly diverge. Conversely, Figure 9 shows that the blank space ratio exhibits a greater variability and it is less susceptible to the insertion of very long lines of legitimate code. For instance, Figure 9b reports the behavior for the package **Vue**, which is characterized by an “irregular” development process. As shown, the blank space ratio increases by roughly 21% between releases 2.6.14 and 2.7.0-alpha.1 due to legitimate stylistic and structural modifications, i.e., the introduction of templates for graphical rendering of web pages containing many blank spaces. However, the injected payload causes a pronounced rise (i.e., approximately 35%), which stands clearly apart from typical evolution-induced fluctuations. The metric is even more expressive for projects with dense release cycles, such as **Angular**, where the large number of versions (> 300) amplifies deviation patterns and makes the anomaly evident, as depicted in Figure 9a. The blank space ratio remains also effective for slow-paced projects, such as **Superstruct** and **Redom**. These repositories show small and incremental modifications over time, making code changes appear more visible in the metric trend.

Overall, our approach was able to correctly identify the injected payload in 70% of the 100 injected npm packages and it only triggered false positives for 73 out of the 37,462 analyzed files. The main cause of false positives is the use of formatting patterns, which can be ascribed to well-known templates. For instance, some packages (e.g., **akkeris** and **d4**) include the description

Table 3: Impact of the trojan on indicators based on encodings and characters for three versions of four representative packages.

Angular				Vue			
Version	Homoglyphs	Chars	Encodings	Version	Homoglyphs	Chars	Encodings
v14.2.9	0	3,589	0	v3.2.43	0	8,732	0
v14.2.10	0	3,589	0	v3.2.44	0	8,321	0
v14.2.11	0	4,341	0	v3.2.45	0	9,822	0
v14.2.11*	4	4,683	4	v3.2.45*	4	9,916	4

Redom				Superstruct			
Version	Homoglyphs	Chars	Encodings	Version	Homoglyphs	Chars	Encodings
v3.25.0	0	3,281	0	v0.6.0	0	696	0
v3.26.0	0	3,281	0	v0.6.0	0	541	0
v3.27.0	0	3,310	0	v0.7.0	0	1,458	0
v3.27.0*	4	4,404	4	v0.7.0*	4	1,790	4

of functionalities through textual drawings and diagrams that embed a large number of blank spaces, causing a legitimate increase in the blank space ratio. Besides **figlet** exploits ASCII art to present its functionalities to implement a “banner” that heavily influences the count of characters and spaces. To partially mitigate this issue, the blank space ratio should be computed by ignoring comments in the code or by identifying drawings containing characters. Despite an overhead for “sanitizing” the `.js` files, this could open up to threats cloaking payload in comments or ASCII-intensive assets, e.g., Base64-encoded attack routines [7].

7.2 Homoglyphs

As expected, the number of encodings confirmed as the most sensitive indicator for spotting the presence of trojan source leveraging homoglyphs. In fact, we observed a generalized absence of “alien” characters, i.e., we found that samples using alternative encodings are almost equal to 0 throughout the development history of the **npm** packages in our dataset.

For the case of the 100 packages trojanized via homoglyphs, such characters were extremely rare: out of 41,903 tags, only 156 versions from 10 projects (i.e., 0.3% of tags) contained characters requiring additional or uncommon encodings. Table 3 shows a compact snapshot on the behavior of encodings for four representative **npm** packages in our dataset. Despite differences in the development process, rules, and habits, each package consistently exhibits 0 homoglyphs. The only exception is the last version when plagued with the hidden trojan payload, which has been denoted in bold and with an asterisk. Here, the value of the metric increases to 4, providing a clear indication of manipulation. The table also reports the number of characters for the various source files composing a release. As shown, the metric is too general to offer a reliable mechanism to discriminate whether an increasing amount of entries should be ascribed to additional functionalities or malicious data. Our framework also generated false positives for 5 packages in our dataset. By manually inspecting them, we found three main causes: mathematical symbols, localization files, and potential (benign) copy-pastes. Specifically, the first affects **Fraction.js** and **num2fraction** that contain algorithms or numerical/mathematical operations, e.g., the developer inserted comments describing the implemented functionalities. The second concerns **Mastodon** and **Rocket.Chat** where text localization (i.e., translation) is offered for some components, implying the presence of characters with different encodings. The third has been observed in **Bootstrap** and it is probably due to copy-pasting from files in other

formats, such as PDFs, which introduce non-visible characters.

Therefore, our preliminary analysis showcased that the evolution of encodings can be considered a valid metric for detecting threats leveraging homoglyphs. Yet, the number of false positives could be non-negligible for packages requiring mathematical symbols or gathering developers from different countries (e.g., those using cyrillic characters). A possible improvement may analyze the abstract syntax tree to discriminate whether homoglyphs have been used in functions or variable names.

8 Conclusions

In this paper, we investigated a prime mechanism for the identification of trojans directly injected in source code via blank spaces and homoglyphs to impair manual software review and inspection practices. To this aim, we investigated seven metrics commonly deployed for code analysis and we introduced three ad-hoc indicators. Our analysis targeted 2,800 `npm` packages and revealed that metrics capturing standard code evolution behaviors (e.g., lines of code) are not effective to spot malicious payloads. Modern repositories characterized by a fast-paced development process require more precise indicators, such as the presence of an abnormal burst of blank spaces. Results highlight the need for lightweight and scalable tools for processing large code bases, which characterize modern software supply chains.

However, this work is still preliminary and has some limitations. First, even if we used 100 realistic `npm` packages, we evaluated our approach on synthetic malicious releases. Since the attack template is known *a priori*, the design of the metrics could lead to an array of highly-specialized indicators. Moreover, an attacker could employ more sophisticated offensive techniques, hindering the identification process offered by an approach based on evolutionary metrics. Additionally, we considered only JavaScript, thus whether our idea can be generalized to different languages requires further investigation.

Future research aims at refining our approach, especially to go beyond the aforementioned limitations. Moreover, part of our ongoing research is devoted to explore automated detection strategies, especially to deploy repair mechanisms for sanitizing code without requiring human intervention.

Acknowledgments

This work was partially supported by Project SERICS (PE00000014) under the NRRP MUR program funded by the EU - NGEU.

Declaration on Generative AI

The authors have not employed any Generative AI tools.

References

- [1] Aikido. RATatouille: A Malicious Recipe Hidden in rand-user-agent. <https://www.aikido.dev/blog/catching-a-rat-remote-access-trojan-rand-user-agent-supply-chain-compromise>.
- [2] Ömer Aslan Aslan and Refik Samet. A Comprehensive Review on Malware Detection Approaches. *IEEE Access*, 8, 2020.

- [3] Nicholas Boucher and Ross Anderson. Trojan Source: Invisible Vulnerabilities. In *Proceedings of the 32nd USENIX Security Symposium*, 2023.
- [4] Emanuele Buchicchio, Luca Grilli, Eros Capobianco, Salvatore Cipriano, and Diego Antonini. Invisible Supply Chain Attacks Based on Trojan Source. *Computer*, 55(10):18–25, 2022.
- [5] Julian Busch, Anton Kocheturov, Volker Tresp, and Thomas Seidl. NF-GNN: Network Flow Graph Neural Networks for Malware Detection and Classification. In *Proceedings of the 33rd International Conference on Scientific and Statistical Database Management*, page 121–132, 2021.
- [6] Luca Caviglione, Michał Choraś, Igino Corona, Artur Janicki, Wojciech Mazurczyk, Marek Pawlicki, and Katarzyna Wasielewska. Tight Arms Race: Overview of Current Malware Threats and Trends in Their Detection. *IEEE Access*, 9:5371–5396, 2020.
- [7] Luca Caviglione and Wojciech Mazurczyk. Never Mind the Malware, Here’s The Stegomalware. *IEEE Security & Privacy*, 20(5):101–106, 2022.
- [8] Yuxiao Chen, Jingzheng Wu, Xiang Ling, Changjiang Li, Zhiqing Rui, Tianyue Luo, and Yanjun Wu. When Large Language Models Confront Repository-level Automatic Program Repair: How Well They Done? In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pages 459–471, 2024.
- [9] Elhaam Debas, Norah Alhumam, and Khaled Riad. Unveiling the Dynamic Landscape of Malware Sandboxing: A Comprehensive Review. *International Journal of Advanced Computer Science and Applications*, 15(3), 2024.
- [10] Dharani Devadiga, Gordon Jin, Bisti Potdar, Hankyu Koo, Andrew Han, Anusha Shringi, Angad Singh, Kinjal Chaudhari, and Saurav Kumar. Gleam: GAN and LLM for Evasive Adversarial Malware. In *2023 14th International Conference on Information and Communication Technology Convergence*, pages 53–58, 2023.
- [11] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *28th Annual Network and Distributed System Security Symposium*, 2021.
- [12] Robert Dyer, Hoan Anh Nguyen, Hridesh Rajan, and Tien N. Nguyen. Boa: Ultra-Large-Scale Software Repository and Source-Code Mining. *ACM Trans. Softw. Eng. Methodol.*, 25(1), 2015.
- [13] Jintao Gu, Hongliang Zhu, Zewei Han, Xiangyu Li, and Jianjin Zhao. GSEDroid: GNN-based Android Malware Detection Framework Using Lightweight Semantic Embedding. *Computers & Security*, 140:103807, 2024.
- [14] Badis Hammi and Sherali Zeadally. Software Supply-Chain Security: Issues and Countermeasures. *Computer*, 56(7):54–66, 2023.
- [15] Panagiotis Kintis, Najmeh Miramirkhani, Charles Lever, Yizheng Chen, Rosa Romero-Gómez, Nikolaos Pitropakis, Nick Nikiforakis, and Manos Antonakakis. Hiding in Plain Sight: A Longitudinal Study of Combosquatting Abuse. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 569–586, 2017.
- [16] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy*, pages 1509–1526, 2023.
- [17] Jonathan AP Marpaung, Mangal Sain, and Hoon-Jae Lee. Survey on Malware Evasion Techniques: State of the Art and Challenges. In *2012 14th International Conference on Advanced Communication Technology*, pages 744–749, 2012.
- [18] MITRE. MITRE ATT&CK. <https://attack.mitre.org/>.
- [19] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 23–43, 2020.
- [20] Delano Oliveira, Reyndne Santos, Fernanda Madeiral, Hidehiko Masuhara, and Fernando Castor. A Systematic Literature Review on the Impact of Formatting Elements on Code Legibility. *Journal of Systems and Software*, 203:111728, 2023.

- [21] Sonatype, Inc. State of the Software Supply Chain. <https://www.sonatype.com/state-of-the-software-supply-chain/introduction>.
- [22] Zhuoran Tan, Shameem Puthiya Parambath, Christos Anagnostopoulos, Jeremy Singer, and Angelos K. Marnierides. Advanced Persistent Threats Based on Supply Chain Vulnerabilities: Challenges, Solutions, and Future Directions. *IEEE Internet of Things Journal*, 12(6):6371–6395, 2025.
- [23] Tristan F.-R. npm-rank. <https://tristan-f-r.github.io/npm-rank/PACKAGES.html>.
- [24] Duc Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. Towards Using Source Code Repositories to Identify Software Supply Chain Attacks. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 2093–2095, 2020.
- [25] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. What are Weak Links in the npm Supply Chain? In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, pages 331–340, 2022.
- [26] Ahmed Zerouali, Tom Mens, Gregorio Robles, and Jesus M Gonzalez-Barahona. On the Diversity of Software Package Popularity Metrics: An Empirical Study of npm. In *2019 IEEE 26th international conference on software analysis, Evolution and Reengineering*, pages 589–593, 2019.
- [27] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proceedings of the 28th USENIX Security Symposium*, pages 995–1010, 2019.